Funktionsplotter Projekt in Programmieren 2 - SS25

von: *Maxim Walter*

Dokumentation

Inhaltsverzeichnis:

- Einführung
  - Eingabe
  - Anzeige
  - Interna
- 1.Erweiterung
- 2.Erweiterung
- Playground

Einführung

Beachten Sie die Hinweise auf der README.md Datei bei der Ausführung der LVP

Modifizierte Turtle.java
Es wurden drei Methoden hinzugefügt, um das Zeichnen mit der Turtle zu erleichtern. Die Turtle.java muss im Verzeichnis enthalten bleiben

```
public Turtle moveTo(double x, double y) {
    if (state.penDown()) {
        elements.add(new Line(++elementCounter,
        state.x(), state.y(), x, y,
        state.color(), state.width()));
```

```
        }
        state = state.withPosition(x, y);
        return this;
    }
    public Turtle setAngle(double angle) {
        double normalizedAngle = (angle % 360 + 360) % 360;
        state = state.withAngle(normalizedAngle);
        return this;
    }
    public double getAngle() {
        return state.angle();
    }
```

Edit Code

Im Folgenden wird die Funktionsweise des Funktionsplotters beschrieben

Eingabe

Es gibt 4 Möglichkeiten wie ein arithmetischer Ausdruck eingegeben werden kann.

1. Der arithmetische Ausdruck, der eine Variable x enthält, kann als Infix-Notation eingegeben werden:

Expression 1 Eingabe in Infix-Notation Send

Es wurde eingegeben: $2 * x^3 + e^x + x/3 - pi$

Der Infix-Ausdruck (mit Variable) wird vom Parser geparsed und der entsprechende AST-Baum generiert:
BinaryOp[op=SUB, left=BinaryOp[op=ADD, left=BinaryOp[op=ADD, left=BinaryOp[op=MUL, left=Number[value=2.0], right=BinaryOp[op=POW, left=Variable[name=x], right=Number[value=3.0]]], right=BinaryOp[op=POW, left=Constant[constant=E], right=Variable[name=x]]], right=BinaryOp[op=DIV, left=Variable[name=x], right=Number[value=3.0]]], right=Constant[constant=PI]]

Hier der dazugehörige Codeabschnitt

```
    String infix_example = "2 * x ^ 3 + e^x + x / 3 - pi"; //
Expression 1
```

```
    Expr infix_example_expr = Parser.parse(infix_example);
```

Edit Code

2. Der arithmetische Ausdruck, der eine Variable x enthält, kann als UPN-Notation eingegeben werden:

Expression 2 Eingabe in UPN-Notation Send

Es wurde eingegeben: x 2 ^ 4 + sqrt x 3 / ln +

Der UPN-Ausdruck (mit Variable) wird vom Parser geparsed und der entsprechende AST-Baum generiert:
BinaryOp[op=ADD, left=FunctionCall[function=SQRT, argument=BinaryOp[op=ADD, left=BinaryOp[op=POW, left=Variable[name=x], right=Number[value=2.0]], right=Number[value=4.0]]], right=FunctionCall[function=LN, argument=BinaryOp[op=DIV, left=Variable[name=x], right=Number[value=3.0]]]]

Hier der dazugehörige Codeabschnitt

```
    String upn_example = "x 2 ^ 4 + sqrt x 3 / ln +"; //
Expression 2
    Expr upn_example_expr = Parser.parse(upn_example);
```

Edit Code

3. Der arithmetische Ausdruck als Infix-Notation (ohne Variable) kann direkt ausgewertet werden:

Expression 3 Eingabe in Infix-Notation Send

Die direkte Auswertung: log_3(27) + sqrt(9) * e ^ pi = 72,422

Hier der Code:

```
    String infix_example_v2 = "log_3(27) + sqrt(9) * e ^ pi ";
// Expression 3
```

```
    String res_infix_v2 = printExpression(infix_example_v2);
```

Edit Code

Es wurde in der printExpression-Methode der String-Ausdruck geparsed, und das richtige Ergebnisformat ausgegeben (map), wenn die tryEval-Methode ein nicht-leeres Optional zurückgibt. Ansonsten wird der fallbackText ausgegeben.

```
public String printExpression(String exprStr) {
    String fallbackText = "Keine unmittelbare Auswertung
möglich";
    return Parser.parse(exprStr)
                .tryEval()
                .map(res -> String.format("%s = %.3f",
exprStr, res))
                .orElse(fallbackText);
}
```

Edit Code

In der tryEval-Methode wird geschaut, ob der Ausdruck eine Variable hat und entsprechend das Ergebnis als Optional zurückgegeben oder als leeres Optional, wenn der Ausdruck eine Variable enthält.
Der folgende Codeausschnitt ist innerhalb des AST-Sealed-Interfaces:

```
    default boolean containsVar() {
        return switch(this) {
            case Expr.Variable v -> true;
            case
Expr.UnaryOp(ArithmeticTokenizer.Token.Operator op, Expr e) ->
e.containsVar();
            case
Expr.BinaryOp(ArithmeticTokenizer.Token.Operator op, Expr l,
Expr r) -> l.containsVar() || r.containsVar();
            case
Expr.FunctionCall(ArithmeticTokenizer.Token.Function f, Expr
arg) -> arg.containsVar();
```

```
            case Expr.LogBase(double base, Expr arg) ->
arg.containsVar();
            default -> false;
        };
    }

    default Optional<Double> tryEval() {
        return this.containsVar() ?  Optional.empty() :
Optional.of(eval(0)); // Wert für x ist hier irrelevant, da
sowieso keine Variable gegeben ist und das Ergebnis konstant
gleich bleibt
    }
```

Edit Code

4.Der arithmetische Ausdruck als UPN-Notation kann direkt ausgewertet werden:

Expression 4 Eingabe in UPN-Notation Send

Die direkte Auswertung: 45 tan 16 sqrt / = 0,405

In den 4 Beispielausdrücken wurden bereits übliche Funktionen, Konstanten etc. eines wissenschaftlichen Taschenrechners eingebunden.
Nun wird eine Übersicht gegeben der möglichen Funktionalitäten mit den verwendeten Symbolen:

1. **Grundrechenarten**
   - Addition: `+`
   - Subtraktion: `-`
   - Multiplikation: `*`
   - Division: `/`
   - Negation: ~ (Bei Ausdrücken ohne Vorgängeroperand)
2. **Potenz**
   - `^`
3. **Wurzel**
   - `sqrt(NUM)`

4. **Logarithmus**
   - Logarithmus zur Basis 10 (Standard): `log(NUM)`
   - Logarithmus mit beliebiger Basis: `log_BASE(NUM)`
   - Natürlicher Logarithmus: `ln(NUM)`
5. **Trigonometrische Funktionen**
   - Sinus: `sin(NUM)`
   - Kosinus: `cos(NUM)`,
   - Tangenz: `tan(NUM)`
6. **Konstanten**
   - Eulersche Zahl e: `e`
   - Kreiszahl π : `pi`
7. **Sonstiges:**
   - e-Funktion e^x : `exp(NUM)`
   - (Alternativ e^x manuell zusammenbauen mit der Konstante e, jedoch ist `exp(NUM)` präziser, da es auf der Math-Library basiert)

Anmerkung: `NUM` und `BASE` müssen logischerweise bei der Angabe durch konkrete Zahlenwerte ersetzt werden

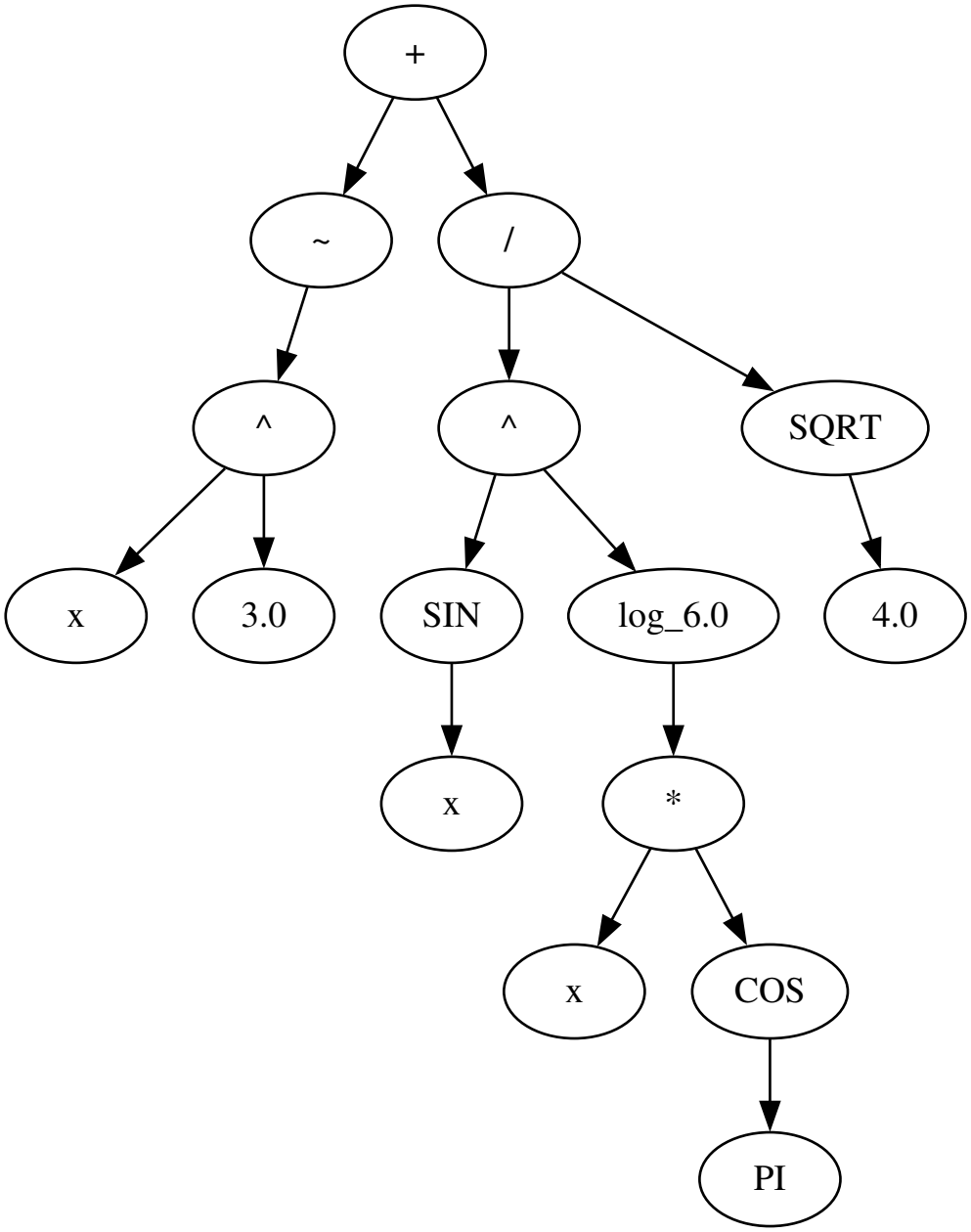
Anzeige

Es wurden bereits im Abschnitt "*Eingabe*" die Ausdrücke in abstrakter Syntax angezeigt

Hier ein weiteres Beispiel: ~x^3 + sin(x)^log_6(x * cos(pi)) / sqrt(4)

Der entsprechende AST-Baum:
BinaryOp[op=ADD, left=UnaryOp[op=NEG, operand=BinaryOp[op=POW, left=Variable[name=x], right=Number[value=3.0]]], right=BinaryOp[op=DIV, left=BinaryOp[op=POW, left=FunctionCall[function=SIN, argument=Variable[name=x]], right=LogBase[base=6.0, expr=BinaryOp[op=MUL, left=Variable[name=x], right=FunctionCall[function=COS, argument=Constant[constant=PI]]]]], right=FunctionCall[function=SQRT, argument=Number[value=4.0]]]]

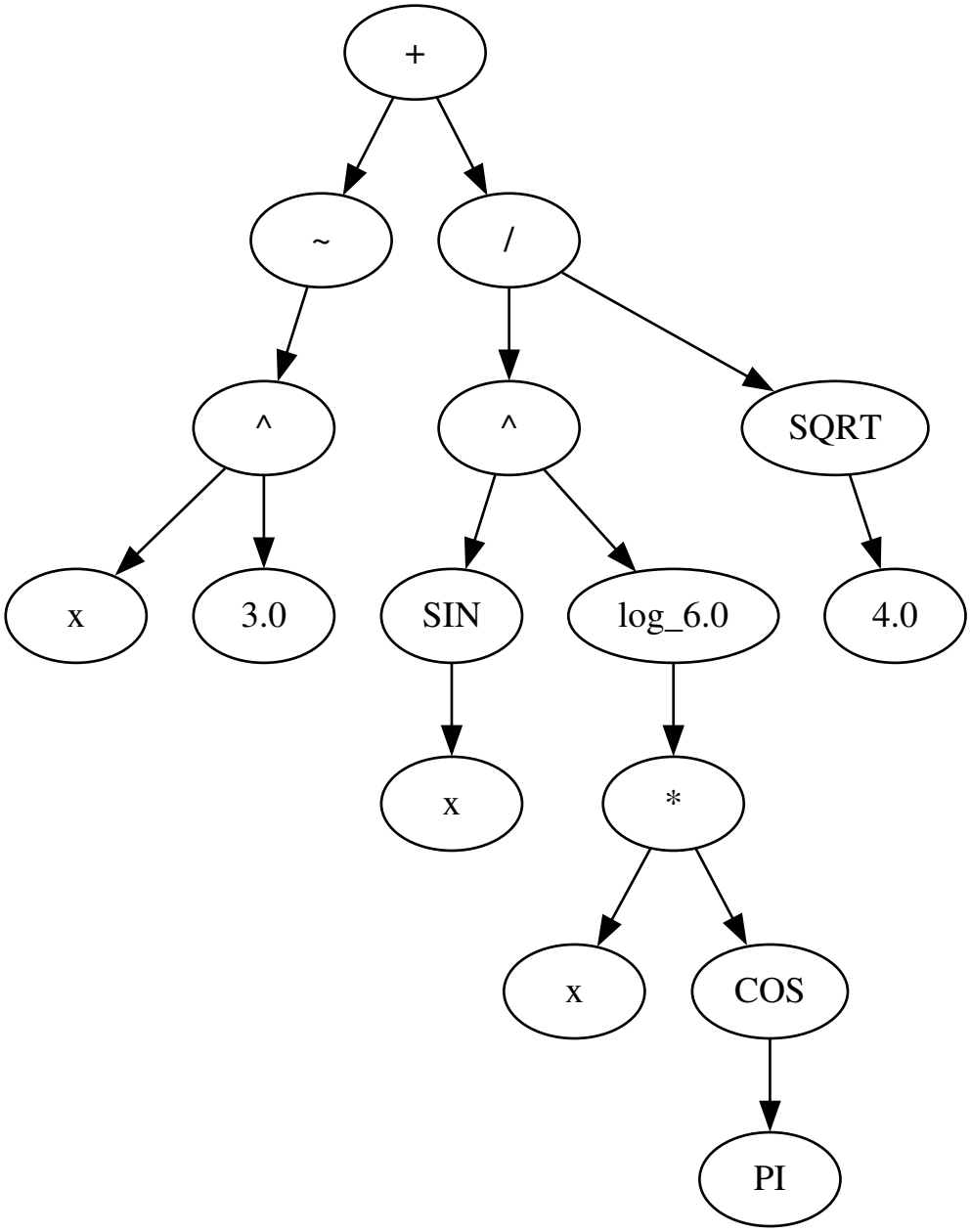
Dieser AST lässt sich mithilfe der DOT-Sprache darstellen:



Die gleiche Funktion nur in UPN-Schreibweise: x 3 ^ ~ x sin x pi cos * log_6 ^ 4 sqrt / +

Der entsprechende AST-Baum aus dem UPN-Ausdruck: UnaryOp[op=NEG, operand=FunctionCall[function=SIN, argument=FunctionCall[function=COS, argument=BinaryOp[op=MUL, left=BinaryOp[op=POW, left=Variable[name=x], right=Number[value=3.0]], right=LogBase[base=6.0, expr=BinaryOp[op=POW, left=Variable[name=x], right=FunctionCall[function=SQRT, argument=BinaryOp[op=ADD, left=Variable[name=x], right=BinaryOp[op=DIV, left=Constant[constant=PI], right=Number[value=4.0]]]]]]]]]]]

Die gleiche Funktion nur in UPN-Schreibweise generiert den gleichen DOT-Graphen:



Die dazugehörige generierte DOT-Syntax sieht folgendermaßen aus:
digraph AST { Node0 [label="+"]; Node1 [label="~"]; Node2 [label="^"]; Node3 [label="x"]; Node4 [label="3.0"]; Node2 -> Node3; Node2 -> Node4; Node1 -> Node2; Node5 [label="/"]; Node6 [label="^"]; Node7 [label="SIN"]; Node8 [label="x"]; Node7 -> Node8; Node9 [label="log_6.0"]; Node10 [label="*"]; Node11 [label="x"]; Node12 [label="COS"]; Node13 [label="PI"]; Node12 -> Node13; Node10 -> Node11; Node10 -> Node12; Node9 -> Node10; Node6 -> Node7; Node6 -> Node9; Node14 [label="SQRT"]; Node15 [label="4.0"]; Node14 -> Node15; Node5 -> Node6; Node5 -> Node14; Node0 -> Node1; Node0 -> Node5; }

Der dazugehörige Code, der aus dem AST-Baum über die `toDot`-Methode die DOT-Syntax generiert ist folgender:

```
class ASTVisualisation {
    private final StringBuilder dot_graph = new
StringBuilder();
    private final Map<Expr, String> node_names = new
HashMap<>();
    private int counter = 0;

    public String toDot(Expr expr) {
        dot_graph.append("digraph AST {\n");
        traverse(expr);
        dot_graph.append("}\n");
        return dot_graph.toString();
    }

    public String traverse(Expr expr) {
        String nodeID = "Node" + counter++;
        node_names.put(expr, nodeID);

        switch (expr) {
            case Expr.Number num -> dot_graph.append("
").append(nodeID).append("
[label=\"").append(num.value()).append("\"];\n");
            case Expr.Variable variable -> dot_graph.append("
").append(nodeID).append("
[label=\"").append(variable.name()).append("\"];\n");
            case Expr.Constant cons -> dot_graph.append("
").append(nodeID).append("
[label=\"").append(cons.constant()).append("\"];\n");
            case
Expr.UnaryOp(ArithmeticTokenizer.Token.Operator op, Expr e) ->
{
                dot_graph.append("  ").append(nodeID).append("
[label=\"").append(op.symbol).append("\"];\n");
                String child_name = traverse(e);
                dot_graph.append("  ").append(nodeID).append("
-> ").append(child_name).append(";\n");
            }
            case
Expr.BinaryOp(ArithmeticTokenizer.Token.Operator op, Expr l,
Expr r) -> {
                dot_graph.append("  ").append(nodeID).append("
[label=\"").append(op.symbol).append("\"];\n");
                String leftId = traverse(l);
                String rightId = traverse(r);
```

```
                dot_graph.append("  ").append(nodeID).append("
-> ").append(leftId).append(";\n");
                dot_graph.append("  ").append(nodeID).append("
-> ").append(rightId).append(";\n");
            }
            case
Expr.FunctionCall(ArithmeticTokenizer.Token.Function func,
Expr arg) -> {
                dot_graph.append("  ").append(nodeID).append("
[label=\"").append(func.name()).append("\"];\n");
                String argId = traverse(arg);
                dot_graph.append("  ").append(nodeID).append("
-> ").append(argId).append(";\n");
            }
            case Expr.LogBase(Double base, Expr arg) -> {
                dot_graph.append("  ").append(nodeID).append("
[label=\"log_").append(base).append("\"];\n");
                String argId = traverse(arg);
                dot_graph.append("  ").append(nodeID).append("
-> ").append(argId).append(";\n");
            }
            case Expr.Parameter p -> dot_graph.append("
").append(nodeID).append("
[label=\"").append(p.name()).append("\"];\n");

        }
        return nodeID;
    }
}
```

Edit Code

Die gesamte Ausführung für diesen Teil ist folgender:

```
    // String example_expr_v3_str = "x^3 + sin(x)^(log_6(x *
cos(pi)) / sqrt(4)"; // FEHLER
    String example_expr_v3_str = "~x^3 + sin(x)^log_6(x *
cos(pi)) / sqrt(4)";
    String example_expr_v3_upn_str = "x 3 ^ ~ x sin x pi cos *
log_6 ^ 4 sqrt / +";
    Expr example_expr_v3 = Parser.parse(example_expr_v3_str);
```

```
    Expr example_expr_v3_upn =
Parser.parse(example_expr_v3_upn_str);
    ASTVisualisation ast_visualisation = new
ASTVisualisation();
    String dot2_syntax =
ast_visualisation.toDot(example_expr_v3);
    String dot3_syntax =
ast_visualisation.toDot(example_expr_v3_upn);


    new Dot().draw(dot2_syntax);


    new Dot().draw(dot3_syntax);
```

Edit Code

Jetzt wird ausgehend von der abstrakten Syntax bzw. dem AST-Baum die Repräsentation in Infix- und UPN-Schreibweise angezeigt

Es wird das vorherige Beispiel verwendet: ~x^3 + sin(x)^log_6(x * cos(pi)) / sqrt(4)

Abstrakte Syntax: `BinaryOp[op=ADD, left=UnaryOp[op=NEG, operand=BinaryOp[op=POW, left=Variable[name=x], right=Number[value=3.0]]], right=BinaryOp[op=DIV, left=BinaryOp[op=POW, left=FunctionCall[function=SIN, argument=Variable[name=x]], right=LogBase[base=6.0, expr=BinaryOp[op=MUL, left=Variable[name=x], right=FunctionCall[function=COS, argument=Constant[constant=PI]]]]], right=FunctionCall[function=SQRT, argument=Number[value=4.0]]]]`

Aus der Abstrakten Syntax wird der Infix-Ausdruck und der UPN-Ausdruck ausgewertet.
Infix: ~x ^ 3.0 + SIN(x) ^ log_6.0(x * COS(pi)) / SQRT(4.0)
UPN: x 3.0 ^ ~ x SIN x pi COS * log_6.0 ^ 4.0 SQRT / +

```
    String infix_notation =
InfixGenerator.generate(example_expr_v3);
    String upn_notation =
```

```
UPNGenerator.generate(example_expr_v3);
```

Edit Code

Ein Funktionsplotter (siehe Playground oder Erweiterungen) kann erstellt werden durch die Objekte der `Plotter`-Klasse.
Dabei können die Min- und Maxwerte der x-Achse und y-Achse gewählt werden

```
    Plotter plotter = new Plotter(-15, 20, -10, 30); // (xMin, xMax, yMin, yMax)
```

Edit Code

Der Plotter wird im Bereich *Playground* angezeigt

Interna

Der AST-Baum wird mit einem `sealed interface` und `records` realisiert. Hier ist ein Teil des Codes zu sehen:

```
sealed interface Expr permits Expr.Number, Expr.Variable,
Expr.UnaryOp, Expr.BinaryOp, Expr.FunctionCall, Expr.Constant,
Expr.LogBase, Expr.Parameter {
    record Number(double value) implements Expr {}
    record Variable(String name) implements Expr {}
    record UnaryOp(ArithmeticTokenizer.Token.Operator op, Expr
operand) implements Expr {}
    record BinaryOp(ArithmeticTokenizer.Token.Operator op,
Expr left, Expr right) implements Expr {}
    record FunctionCall(ArithmeticTokenizer.Token.Function
function, Expr argument) implements Expr {}
    record Constant(ArithmeticTokenizer.Token.Constant
constant) implements Expr {}
    record LogBase(double base, Expr expr) implements Expr {}
    record Parameter(String name) implements Expr {}
    // (...)
```

Edit Code

Der arithmetische Ausdruck wird von einem Tokenizer verarbeitet.
Beispiel:
[Number[value=4.0], ADD, Variable[name=x], DIV, LogBase[base=5.0], OPEN, Number[value=3.0], CLOSE, POW, SIN, OPEN, Variable[name=x], MUL, Number[value=2.0], CLOSE]

```
    List<ArithmeticTokenizer.Token> tokens_example =
ArithmeticTokenizer.tokenize("4 + x / log_5(3) ^ sin(x*2)");
    String tokens_example_str = tokens_example.toString();
```

Edit Code

In der `Parser`-Klasse werden beide Parser (UPN und Infix) kombiniert. Wenn beide Parser fehlschlagen wird eine Exception geworfen

```
    public static Expr parse(String input) {
        List<ArithmeticTokenizer.Token> tokens =
```

```
ArithmeticTokenizer.tokenize(input);
        // Zuerst UPN versuchen
        try {
            validateUPN(tokens);
            return UPNParser.parse(tokens);
        } catch (IllegalArgumentException e1) {
            // Falls UPN fehlschlägt, Infix versuchen
            try {
                return InfixParser.parse(tokens);
            } catch (IllegalArgumentException e2) {
                // UPN und Infix schlägt fehl
                throw new IllegalArgumentException(
                    "Weder gültige UPN noch Infix Notation\n"
+
                    "UPN Fehler: " + e1.getMessage() + "\n" +
                    "Infix Fehler: " + e2.getMessage());
            }
        }
    }
```

Edit Code

1. Erweiterung: Eingabe und Anzeige mehrerer und farblich unterschiedlicher Funktionen

Es können mehrere Funktionsverläufe auf einem Graphen angezeigt werden, jedoch ist nicht die Farbwahl miteinbezogen. Mit jeder neuen Funktion wird eine neue Farbe generiert, indem der aktuelle Farbwert mit dem Goldenen Schnitt inkrementiert wird (durch Modulo im Bereich [0,1] bleiben).
Zuerst wird der aktuelle Farbton aus dem HSL-Farbraum mit einem hohen Sättigungs- und Helligkeitswert versehen, damit die Farben sich deutlich mit Kontrast unterscheiden. Anschließend wird die HSL-Farbe in RGB konvertiert und das berechnete Array aus dem RGB-Farben kann verwendet werden z.B. in `plotMultipleFunctions(Expr, double...)`

```
// ================= Farbgenerator =================
class ColorGenerator {
    private static final float GOLDEN_NUM =
```

```
1.618033988749895f;
    private float hue = 0;

    public int[] getNextColor() {
        hue += GOLDEN_NUM;
        hue %= 1; // Bleibt im Bereich [0,1]

        // HSB zu RGB: Sättigung 90%, Helligkeit 95%
        int rgb = Color.HSBtoRGB(hue, 0.9f, 0.95f);
        return new int[] {
            (rgb >> 16) & 0xFF, // Rot
            (rgb >> 8) & 0xFF,  // Grün
            rgb & 0xFF          // Blau
        };
    }
}
```

Edit Code

Die Graphanzeige kann unter dem Abschnitt `Playground` betrachtet werden.

2. Erweiterung: Funktionen mit Parameter a, deren Werte man bei der Anzeige frei wählen kann und gleichzeitig die verschiedenen Funktionsverläufe zeigt

---

Den Ausdrücken kann zusätzlich der Parameter a versehen werden mit beliebig vielen Werten, indem man beliebig weitere Argumente in die `plotFunctionsWithParams` übergibt

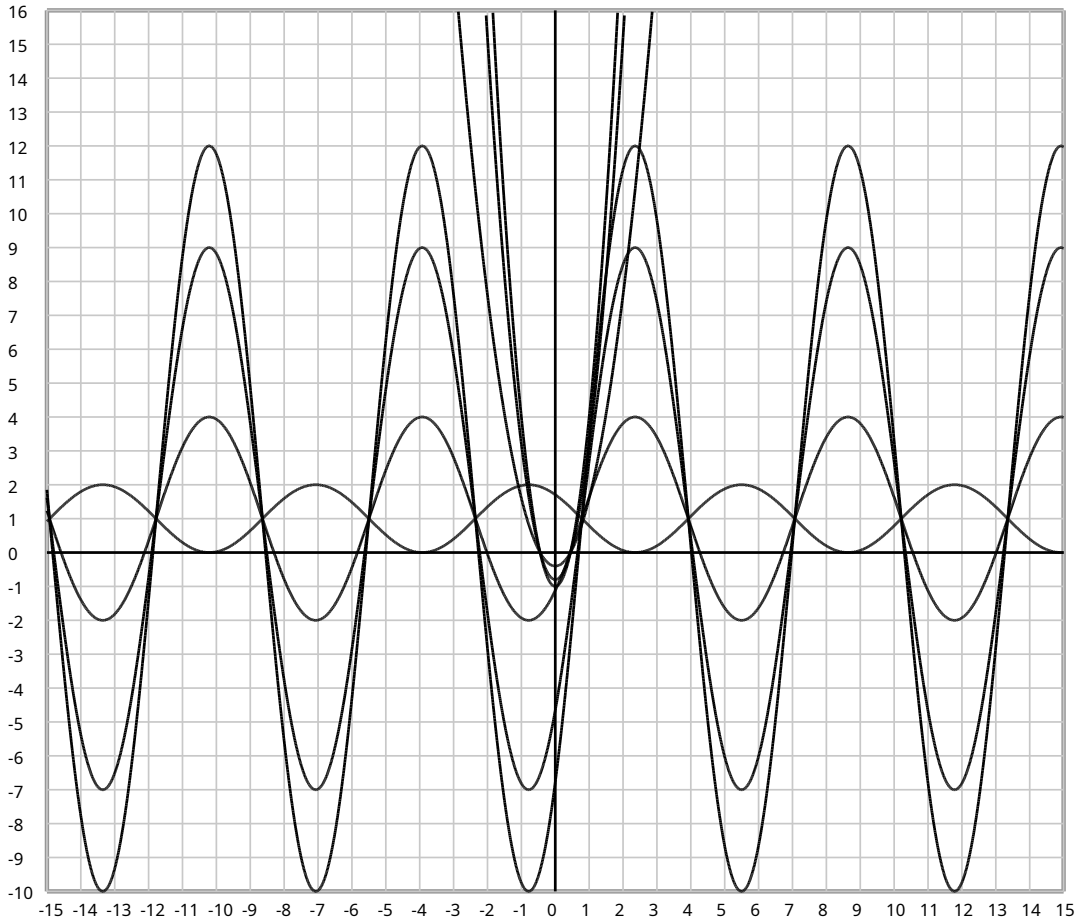
```
    Plotter p2 = new Plotter(-15, 15, -10, 16);
    Expr first_expr_p2 = Parser.parse("a * x^2 - a * 1/5");
    Expr second_expr_p2 = Parser.parse("a * sin(x - pi / 4) +
1");

    p2.redraw();
    p2.plotFunctionWithParams(first_expr_p2, "a", 2.0, 4.0,
5.0);
    p2.plotFunctionWithParams(second_expr_p2, "a", -1.0, 3.0,
```

```
8.0, 11.0);
    p2.startTurtle();
```

Edit Code

Anmerkung: Im Playground sind die Parameterwerte Plotterabhängig und werden für alle verschiedenen Parameterfunktionen verwendet d.h. man kann nicht einzelnd unterschiedliche Parameterwerte für verschiedene Funktionen auswählen. Die Farben werden dort auch angezeigt

Funktionsgraph:



Playground: Der Spaß fängt an!

Funktionseingabe:

Die Funktionen können in Form von Ausdrücken kommagetrennt als Strings angegeben werden.

```
String[] examples = {
    "sin(x) + cos(x)", // Infix mit Funktion
    "a * e^x + 6",
    "4*x^3 + log_4(x)",
    "1/x",
    "45 cos 10 * 9 sqrt +",
    //"3 x 2 ^ * 5 -", // UPN mit Variable
    // "x sin x cos +",   // UPN mit Funktion
    // "(6 + 2) * 2", // Infix mit Klammern
    // "6 + 2 * 5",    // Infix ohne Klammern
    // "6 + (2 * 5)", // Infix mit Klammern
    // "~x+2",
    //"e^2",
    // "1/x",
    // "x^3 - 2 * x",
    // "sin(x)*5",
    // "ln(x)",
    // "e^x",
};
```

Edit Code

Zusätzlich die Eingabe der Parameterwerte für a, die für alle Parameterfunktionen innerhalb dieses Plotters gelten:

```
double[] parameterValues = {
    1.0,
    3.0,
    -1.5,
};
```

Edit Code

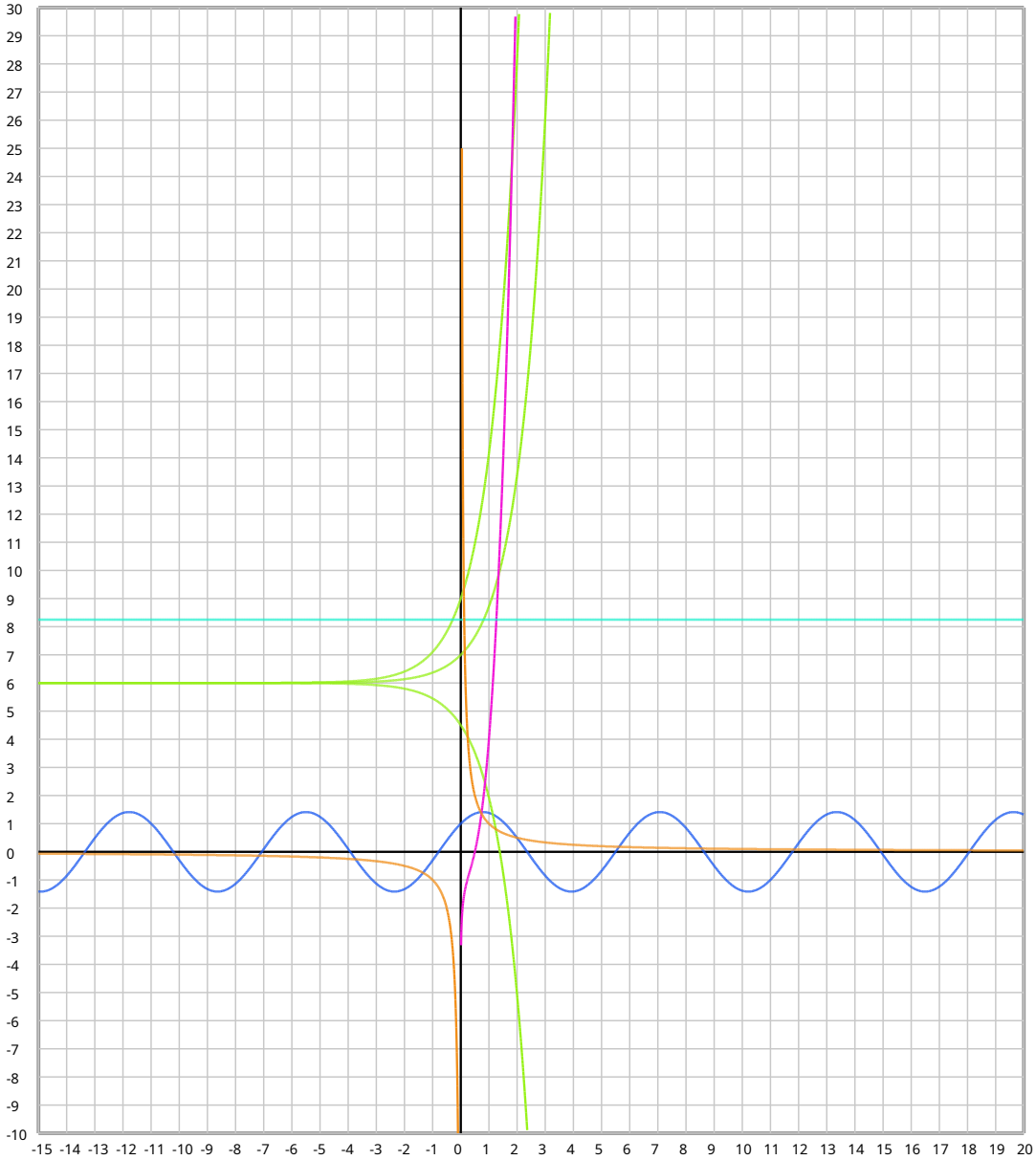
Die Grenzen des Funktionsplotter können angepasst werden:

```
Plotter plotter = new Plotter(-15, 20, -10, 30); // (xMin,
```

```
xMax, yMin, yMax)
```

Edit Code

Funktionsgraph:



Funktionsübersicht:

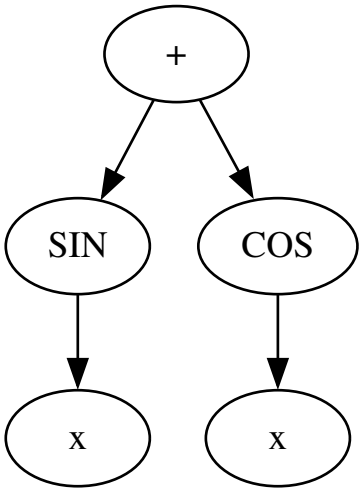
Funktion Nr.1

DOT-Syntax:

digraph AST { Node0 [label="+"]; Node1 [label="SIN"]; Node2 [label="x"]; Node1 -

> Node2; Node3 [label="COS"]; Node4 [label="x"]; Node3 -> Node4; Node0 -> Node1; Node0 -> Node3; }

DOT-Graph:



Direkte Auswertung: Nicht möglich, da der Ausdruck eine Variable enthält.

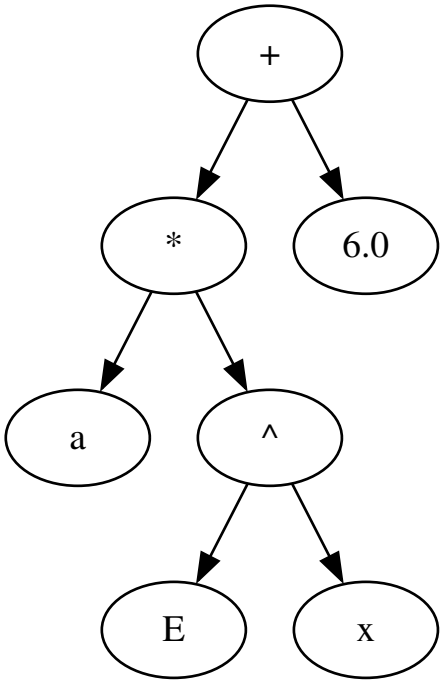
Infix: SIN(x) + COS(x)

UPN: x SIN x COS +

Funktion Nr.2

DOT-Syntax:
digraph AST { Node0 [label="+"]; Node1 [label="*"]; Node2 [label="a"]; Node3 [label="^"]; Node4 [label="E"]; Node5 [label="x"]; Node3 -> Node4; Node3 -> Node5; Node1 -> Node2; Node1 -> Node3; Node6 [label="6.0"]; Node0 -> Node1; Node0 -> Node6; }

DOT-Graph:



Benutzte Parameterwerte: [1.0, 3.0, -1.5]

Direkte Auswertung: Nicht möglich, da der Ausdruck eine Variable oder Parameter enthält.

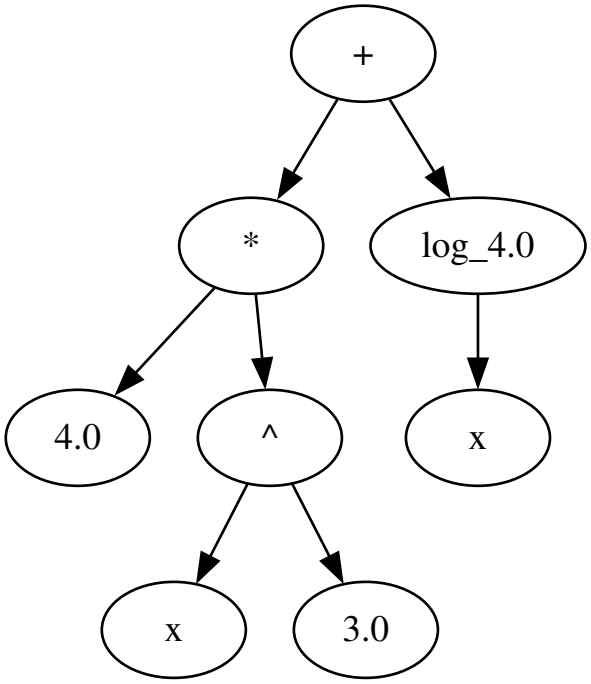
Infix: a * e ^ x + 6.0

UPN: a e x ^ * 6.0 +

Funktion Nr.3

DOT-Syntax:
digraph AST { Node0 [label="+"]; Node1 [label="*"]; Node2 [label="4.0"]; Node3 [label="^"]; Node4 [label="x"]; Node5 [label="3.0"]; Node3 -> Node4; Node3 -> Node5; Node1 -> Node2; Node1 -> Node3; Node6 [label="log_4.0"]; Node7 [label="x"]; Node6 -> Node7; Node0 -> Node1; Node0 -> Node6; }

DOT-Graph:



Direkte Auswertung: Nicht möglich, da der Ausdruck eine Variable enthält.

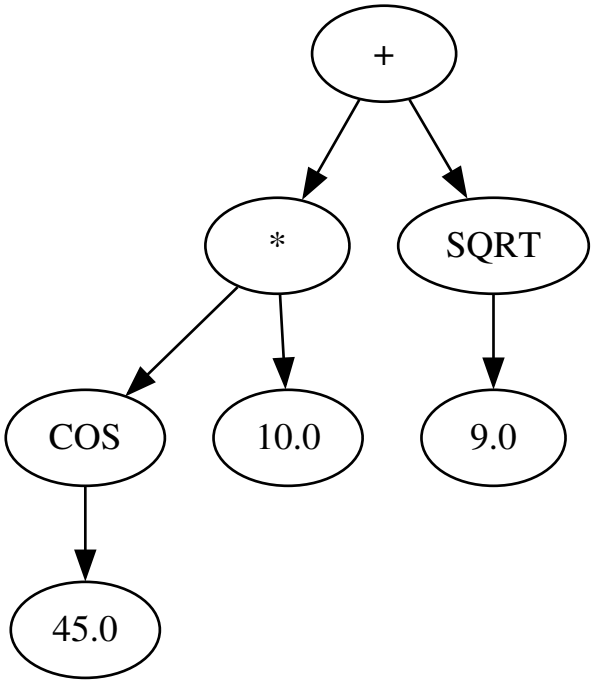
Infix: 4.0 * x ^ 3.0 + log_4.0(x)

UPN: 4.0 x 3.0 ^ * x log_4.0 +

Funktion Nr.4

DOT-Syntax:
digraph AST { Node0 [label="+"]; Node1 [label="*"]; Node2 [label="COS"]; Node3 [label="45.0"]; Node2 -> Node3; Node4 [label="10.0"]; Node1 -> Node2; Node1 -> Node4; Node5 [label="SQRT"]; Node6 [label="9.0"]; Node5 -> Node6; Node0 -> Node1; Node0 -> Node5; }

DOT-Graph:



Direkte Auswertung: 45 cos 10 * 9 sqrt + = 8,253

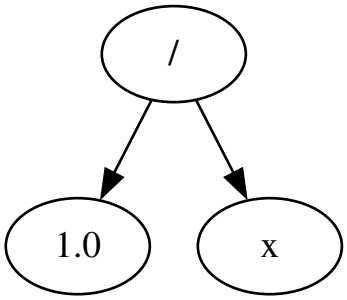
Infix: COS(45.0) * 10.0 + SQRT(9.0)

UPN: 45.0 COS 10.0 * 9.0 SQRT +

Funktion Nr.5

DOT-Syntax:
digraph AST { Node0 [label="/"]; Node1 [label="1.0"]; Node2 [label="x"]; Node0 -> Node1; Node0 -> Node2; }

DOT-Graph:



Direkte Auswertung: Nicht möglich, da der Ausdruck eine Variable enthält.

Infix: 1.0 / x

UPN: 1.0 x /